Assignment 2 – TensorFlow Fundamentals and Model Training

Submit to : Rajash Sir

Submit By : Aakash (R50951)

Objective

Practice TensorFlow tensors, variables, automatic differentiation and gradient-based training by implementing a simple linear regression model.

Dataset / Starting Material

```
A = [
    [1,2],
    [3,4]
]

B = [
    [5,6],
    [7,8]
]

x = 5

y = x**2
print(y)
```

```
25
```

Tasks

Ques 1: Create tensors A and B using the given values.

```
import tensorflow as tf

A = tf.constant(A)
B = tf.constant(B)

print(A)
```

```
tf.Tensor(
[[1 2]
 [3 4]], shape=(2, 2), dtype=int32)
```

```
print(B)
```

```
tf.Tensor(
[[5 6]
 [7 8]], shape=(2, 2), dtype=int32)
```

Ques 2: Perform tensor addition, subtraction, element-wise multiplication and matrix multiplication.

1. Addition

```
add = tf.add(A,B)
print((add))

tf.Tensor(
[[ 6  8]
 [10 12]], shape=(2, 2), dtype=int32)
```

1. sub

```
sub = tf.subtract(A,B)
print(sub)

tf.Tensor(
[[-4 -4]
 [-4 -4]], shape=(2, 2), dtype=int32)
```

1. multiplication

```
multi = tf.multiply(A,B)
print(multi)

tf.Tensor(
[[ 5 12]
 [21 32]], shape=(2, 2), dtype=int32)
```

1. matrix multiplication

```
mat = tf.matmul(A,A) # use matmut : for matrix multi
print(mat)

tf.Tensor(
[[ 7 10]
 [15 22]], shape=(2, 2), dtype=int32)
```

Ques 3. Display the shape, rank and data type of tensor A.

```
print(A.shape)

(2, 2)

print(A.ndim)

2
```

```
tf.rank(A)
```

```
<tf.Tensor: shape=(), dtype=int32, numpy=2>
```

```
print(A.dtype)
```

```
<dtype: 'int32'>
```

Ques 4. Create a TensorFlow Variable and demonstrate how its value can be updated.

```
var = tf.Variable(A)
print(var)
```

```
<tf.Variable 'Variable:0' shape=(2, 2) dtype=int32, numpy=
array([[1, 2],
       [3, 4]], dtype=int32)>
```

```
var = var + 1
print(var)
```

```
tf.Tensor(
[[2 3]
 [4 5]], shape=(2, 2), dtype=int32)
```

Ques 5. Use tf.GradientTape() to calculate the derivative of $y = x^2$ for $x = 5$.

```
import tensorflow as tf

x = tf.Variable(5.0)

with tf.GradientTape() as tape:
  tape.watch(x)
  y = x**2
  dy = tape.gradient(y,x)

print(x.numpy())
print(y.numpy())
print(dy.numpy())
```

```
5.0
25.0
10.0
```

Ques 6. Implement a simple linear regression model using $y = wx + b$.

```
# Generate some synthetic data
x_train = tf.constant([1.0, 2.0, 3.0, 4.0, 5.0], dtype=tf.float32)
y_train = tf.constant([2.0, 4.0, 5.0, 4.5, 6.0], dtype=tf.float32) #
Target values
```

Ques 7. Initialize the trainable parameters w and b.

```
# Initialize trainable parameters w and b
w = tf.Variable(0.5, dtype=tf.float32, name='weight')
b = tf.Variable(0.0, dtype=tf.float32, name='bias')

print(f"Initial weight (w): {w.numpy()}")
print(f"Initial bias (b): {b.numpy()}")
```

```
Initial weight (w): 0.5
Initial bias (b): 0.0
```

Ques 8. Calculate predictions and Mean Squared Error (MSE) loss.

```
# Calculate predictions
y_predicted = w * x_train + b

# Calculate Mean Squared Error (MSE) loss
loss = tf.reduce_mean(tf.square(y_predicted - y_train))

print(f"Initial predictions: {y_predicted.numpy()}")
print(f"Initial MSE Loss: {loss.numpy()}")
```

```
Initial predictions: [0.5 1.  1.5 2.  2.5]
Initial MSE Loss: 8.399999618530273
```

Ques 9. Use tf.GradientTape() to calculate gradients of the loss with respect to w and b.

```
with tf.GradientTape() as tape:
  # Re-calculate predictions and loss inside the tape scope to track
gradients
  y_predicted = w * x_train + b
  loss = tf.reduce_mean(tf.square(y_predicted - y_train))

# Calculate gradients of the loss with respect to w and b
grad_w, grad_b = tape.gradient(loss, [w, b])

print(f"Gradient of loss with respect to w: {grad_w.numpy()}")
print(f"Gradient of loss with respect to b: {grad_b.numpy()}")
```

```
Gradient of loss with respect to w: -18.200000762939453
Gradient of loss with respect to b: -5.599999904632568
```

Ques 10. Update w and b using gradient descent for multiple epochs.

```
epochs = 100
learning_rate = 0.01

print(f"Starting training with {epochs} epochs and learning rate
{learning_rate}")
```

```
for epoch in range(epochs):
  with tf.GradientTape() as tape:
    # Recalculate predictions and loss inside the tape scope for each
epoch
    y_predicted = w * x_train + b
    loss = tf.reduce_mean(tf.square(y_predicted - y_train))

  # Calculate gradients
  grad_w, grad_b = tape.gradient(loss, [w, b])

  # Update weights and biases using gradient descent
  w.assign_sub(learning_rate * grad_w)
  b.assign_sub(learning_rate * grad_b)

  if (epoch + 1) % 10 == 0:
    print(f"Epoch {epoch + 1}, Loss: {loss.numpy():.4f}, w:
{w.numpy():.4f}, b: {b.numpy():.4f}")

print(f"\nTraining complete.\nFinal w: {w.numpy():.4f}\nFinal b:
{b.numpy():.4f}")
```

```
Starting training with 100 epochs and learning rate 0.01
Epoch 10, Loss: 0.7780, w: 1.2090, b: 0.2514
Epoch 20, Loss: 0.6925, w: 1.2439, b: 0.3143
Epoch 30, Loss: 0.6675, w: 1.2339, b: 0.3630
Epoch 40, Loss: 0.6444, w: 1.2214, b: 0.4092
Epoch 50, Loss: 0.6229, w: 1.2090, b: 0.4539
Epoch 60, Loss: 0.6027, w: 1.1970, b: 0.4970
Epoch 70, Loss: 0.5838, w: 1.1855, b: 0.5388
Epoch 80, Loss: 0.5662, w: 1.1743, b: 0.5791
Epoch 90, Loss: 0.5498, w: 1.1635, b: 0.6181
Epoch 100, Loss: 0.5344, w: 1.1531, b: 0.6558

Training complete.
Final w: 1.1531
Final b: 0.6558
```

Ques 11. Display the loss during training and the final values of w and b.

```
# loss we can see in prevous step
print(f"\nTraining complete.\nFinal w: {w.numpy():.4f}\nFinal b:
{b.numpy():.4f}")
```

```


Training complete.
Final w: 1.1531
Final b: 0.6558
```

Ques 12. Use the trained model to make a prediction for a new input value.

```
new_x = tf.constant(7.0, dtype=tf.float32)
predicted_y_new = w * new_x + b

print(f"For a new input x = {new_x.numpy()}, the predicted y is:
{predicted_y_new.numpy():.4f}")
```

```
For a new input x = 7.0, the predicted y is: 8.7273
```

Ques 13. Plot actual values and predicted values and write a short observation.

```
import matplotlib.pyplot as plt

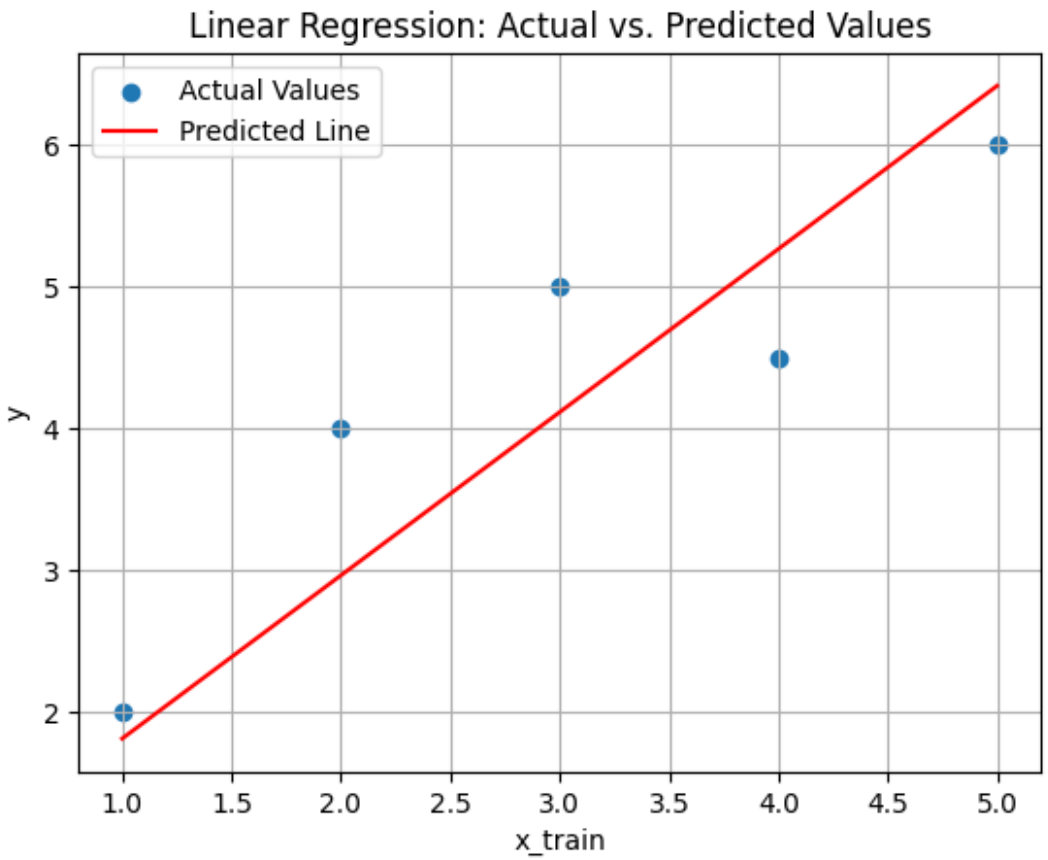
# Generate predictions for the training data using the final w and b
y_train_predicted = w * x_train + b

# Plot actual values
plt.scatter(x_train.numpy(), y_train.numpy(), label='Actual Values')

# Plot predicted values (regression line)
plt.plot(x_train.numpy(), y_train_predicted.numpy(), color='red',
label='Predicted Line')

plt.xlabel('x_train')
plt.ylabel('y')
plt.title('Linear Regression: Actual vs. Predicted Values')
plt.legend()
plt.grid(True)
plt.show()

print("\nObservation:\n")
print("The plot shows that the trained linear regression model,
represented by the red line, provides a reasonably good fit to the
actual data points. The line generally follows the trend of the
scattered points, indicating that the model has learned a linear
relationship between x and y. While not a perfect fit (as indicated by
the final loss), it captures the overall pattern in the data.")
```




```
Observation:

The plot shows that the trained linear regression model, represented by the red line, provides a reasonably good fit to the actual data points. The line generally follows the trend of the scattered points, indicating that the model has learned a linear relationship between x and y. While not a perfect fit (as indicated by the final loss), it captures the overall pattern in the data.
```

Questions to Answer

Ques 1. What is a TensorFlow tensor?

Ans : tensor is fundamental data structure use in Tensorflow(Deep learning). It is multi-dim array structure used to store and manipulation data. It is immutable by default that mean we can't change after create tansor.

Ques 2. What is the difference between a Tensor and a Variable?

Ans : Difference b/t tensor and variable

tensor :

- create like that tf.Tensor
- it is immutable(can't change after create a tensor)
- it is not trainable because in dl during training need to changes
- it use use to cantain data/array, but it optimize cup process

Variable

- it create like that : tf.Variable
- it is mutable that mean we can change according to requrement
- it is trainable so we can use in dl
- we can track using GradientTape

Ques 3. What does tf.GradientTape() do?

Ans : It is a Tensorflow api that able to automatic differentiation. It records operations performed during a forward pass to compute the gradients of a target(usually the loss function) with some perticular(main or one by one) tensor(trainable variable w/b)

how it work

1. recording operations
   - when execte the **with tf.GradientTape() as tape** block
   - tape obj record all operations
   - by default operation happen with **tf.Variable**
2. forward pass : after calculate gradient and find prediction and find loss finction
3. backward pass ; ater forward pass then computing Gradient, it have 2 thing
   - target :- tensor whose gradient want to cal. according to loss
   - sourse :- a list of tensor we want to cal gradient

Ques 4. What is a gradient?

Ans : A gradient is vector that contain/hold all partial derivatives(dy/dx) of a function

Ques 5. Why do we use a loss function?

Ans : loss function is a way to measures prediction of our model Vs actual

Ques 6. What is the purpose of gradient descent?

Ans : the purpose of Gradient descent is to minimize the loss/error of the deeplearning model, it continouse change w & b

Ques 7. What happens to the loss as training progresses?

Ans : In training generaly loss decrese because model learn pattern form data and imporve prediction

Thank you